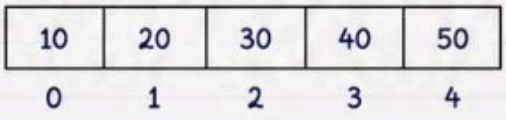
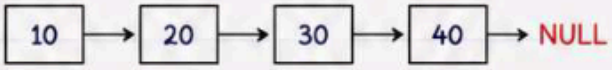
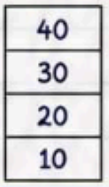
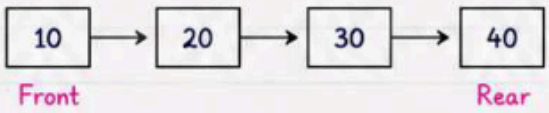
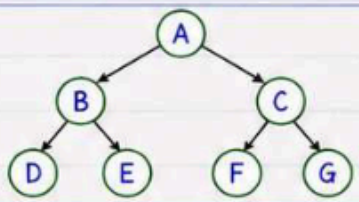
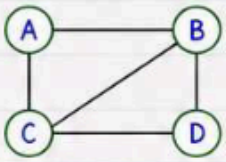
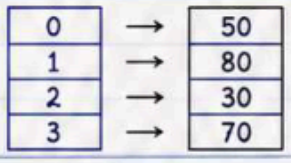
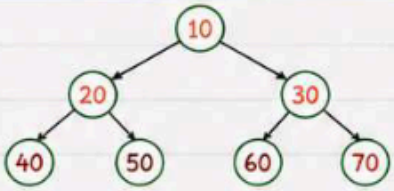
Data Structures

Definition

A data structure is a way of organizing and storing data so that it can be used efficiently. It helps in optimizing the performance of programs by allowing efficient access, modification, and processing of data.

All Data Structures Overview

Data Structure	Diagram / Representation	Description
1. Array		Stores elements of the same type in contiguous memory locations.
2. Linked List		Stores elements in nodes. Each node points to the next node.
3. Stack (LIFO)		Follows LIFO (Last In, First Out) principle. Insertion & deletion at the top.
4. Queue (FIFO)		Follows FIFO (First In, First Out) principle. Insertion at rear, deletion at front.
5. Tree		Hierarchical structure with a root node and zero or more subtrees.
6. Graph		Collection of nodes (vertices) connected by edges.
7. Hash Table (Hash Map)		Stores key-value pairs using a hash function for fast access.
8. Heap		A special tree-based structure (Min-Heap or Max-Heap).

Pro Tip: Understanding the right data structure is the key to writing efficient and optimized code. ☆

70 DSA Interview Questions

Complete Java Solutions Handbook

Every problem from the 70 DSA Interview Questions carousel, in the same order, with a clean Java solution, the core idea/approach, and time & space complexity — ready to revise before your next coding interview.

Chapter	# Problems
Arrays	10
Strings	10
Linked List	10
Stack & Queue	10
Trees	15
Graphs	5
Dynamic Programming	5
Searching + Sorting	10

ARRAYS

10 Java Solutions — Interview Ready

1. Two Sum

Idea: *Use HashMap to store value -> index.*

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (map.containsKey(need)) return new int[]{map.get(need), i};
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

2. Best Time to Buy & Sell Stock

Idea: *Track minimum price seen so far.*

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, profit = 0;
        for (int p : prices) {
            minPrice = Math.min(minPrice, p);
            profit = Math.max(profit, p - minPrice);
        }
        return profit;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)















